

POSSESSION OF MOBILES IN EXAM IS UFM PRACTICE.

Name _____

Enrollment No. _____

Jaypee Institute of Information Technology, Noida

**T2 Examination, Even 2024
B.Tech VI Semester**

**Course Title : Cloud based enterprise systems
Course Code : 15B22CI521**

**Maximum Time : 01 Hr
Maximum Marks : 20**

Course description:

CO1	Differentiate between public, private, and hybrid clouds
CO2	Develop enterprise applications based JavaScript, Node.js, etc.
CO3	Develop web service based solutions by using REST, JSON, etc.
CO4	Examine emerging technologies in cloud environment
CO5	Evaluate the performance of different public cloud platforms e.g. GAE, AWS and Azure.
CO6	Design and deploy enterprise applications on one of the cloud service providers, i.e. AWS or Microsoft Azure

Note: Attempt all the questions

Q1. Design a JSON schema for validating data representing employee information within a company. The schema should ensure that each employee record contains fields for name, age, department, and salary. Additionally, the schema should enforce specific data types and constraints, such as ensuring that the age is a positive integer and the salary is non-negative number. Write sample JSON data that conforms to this schema, and also provide examples of invalid data that would fail the schema validation.

[CO3 (Apply level), 3 marks]

Q2. Suppose we have two collections employees {_id: ObjectId, name: String, department_id: ObjectId, salary: Number}, and departments {_id: ObjectId, name: String, location: String}. Each document in the 'employees' collection contains a 'department_id' field that corresponds to a document in the 'departments' collection. Perform the following task:

a) Join collections query: Write a MongoDB query that utilizes the '\$lookup' operator to join the 'employees' and 'departments' collections on the 'department_id' field. The result should be a new collection where each employee document includes the corresponding department document.

b) Aggregate the results: After joining the collections, write a MongoDB query that uses the '\$group' operator to calculate the total salary of each department. The result should be a new collection where each document includes a department and the total salary of that department.

[CO4 (Analyze level), 3 marks]

Q3. Develop an e-commerce application with Dockerization consisting of three layers: frontend, backend, and database. The application is built using Node.js for both the frontend and backend, and MongoDB serves as the database. Perform following tasks:

a) Frontend layer (Node.js):

- i) Develop a simple Node.js program with a route to serve the home page
- ii) Create a Dockerfile for the frontend layer using Node.js as the base image. Ensure the Dockerfile installs dependencies and sets up the environment for the frontend application. Expose the 3000 ports for the frontend to communicate with the outside world.
- iii) Connects to the network named 'JIIT'.
- iv) Attach a volume named 'FileserverN' to persist static files or assets used by the frontend application.

[CO2 (Apply level), 3 marks]

b) Backend layer (Node.js with MongoDB):

- i) Develop a Node.js program with all the CRUD operations for a collection product {type: String, price: Number, description: String, category: String}.
- ii) Create a Dockerfile for the backend layer using Node.js as the base image. Include steps to install dependencies, setup the environment, and connect to MongoDB. Expose the 5000 port for the backend API.
- iii) Ensure error handling and validation techniques are implemented within the Dockerized backend.
- iv) Connects to the network named 'JIIT'.
- v) Attach a volume named 'FileserverN' to persist static files or assets used by the frontend application.

[CO3 (Apply level), 6 marks]

c) Database layer (MongoDB):

- i) Create a Dockerfile for the database layer using the official MongoDB image. Includes necessary configurations or initialization scripts to set up the database environment. Expose the MongoDB port to allow communication with the backend layer.
- ii) Connects to the network named 'JIIT'.
- iii) Attach a volume named 'FileserverN' to persist static files or assets used by the frontend application.

[CO4 (Analyse level), 2 marks]

d) Docker compose configuration:

- i) Create a Docker compose configuration file ('docker-compose.yml') to orchestrate the containerized environment.
- ii) Define services for the frontend, backend, and database layers within the Docker Compose configuration.
- iii) Ensure that all containers are connects to the network named 'JIIT' for seamless communication.
- iv) Utilize persistent volumes for the database.

[CO6 (Create level), 3 marks]